

CRYPTOGRAPHY COURSE

Debugging & Optimisation Task

CO2-AT-3

NAME:S.DHIVYASRI

REG NO:192511246

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES

Implementation)

A Python program implementing DES and 3DES encryption produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).

C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

ANSWER:

```
from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad, unpad
import time

# 24-byte (192-bit) key for 3DES
key = b'123456789012345678901234'

# Plaintext
plaintext = b"Secure Communication using 3DES"

print("Original Plaintext :", plaintext.decode())

# Create cipher
cipher = DES3.new(key, DES3.MODE_ECB)

# Encryption
start = time.time()

ciphertext = cipher.encrypt(pad(plaintext, DES3.block_size))

end = time.time()

print("\nEncrypted Ciphertext (Hex):")
print(ciphertext.hex())

print("\nEncryption Time :", (end-start)*1000, "ms")

# Decryption
cipher = DES3.new(key, DES3.MODE_ECB)

start = time.time()
```

Console

```
Run started
Initializing environment
Installing packages
Running code
Original Plaintext : Secure Communication using 3DES

Encrypted Ciphertext (Hex):
50ca6ca3cf94d338ec09f8e2f59c59f462f327ec68b0a808777bb30e2b5df394

Encryption Time :
0.9999275207519531 ms

Decrypted Plaintext :
Secure Communication using 3DES

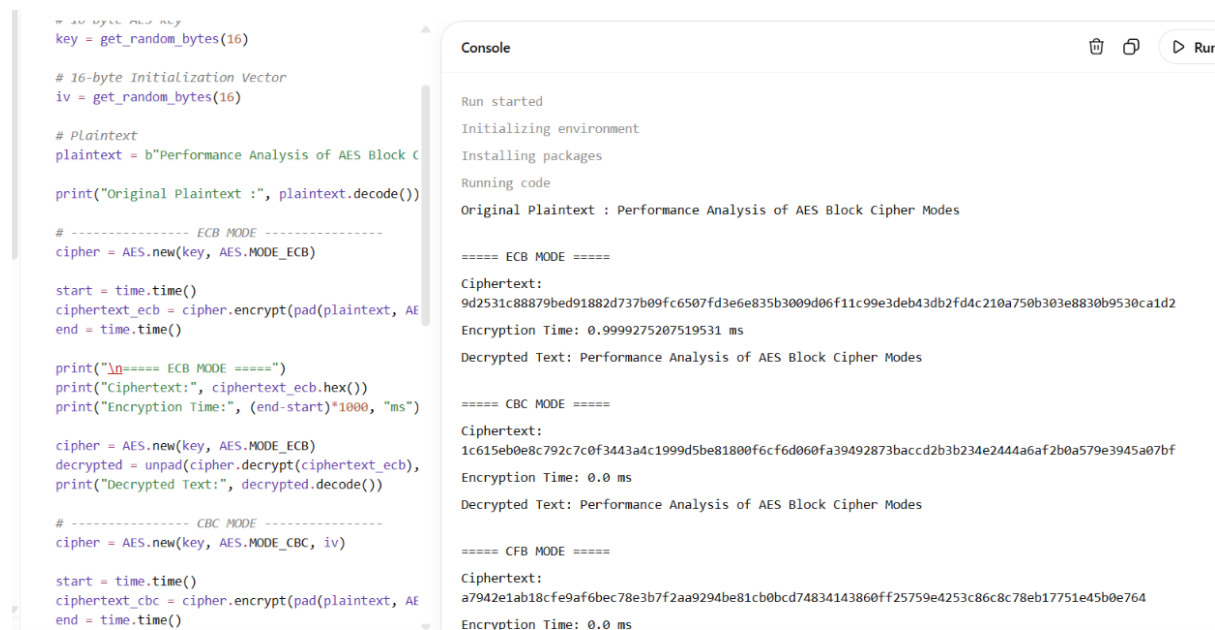
Decryption Time :
0.0 ms
Run completed in 3906.799999523163ms
```

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements AES-like block cipher modes (ECB, CBC, CFB, OFB) but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

ANSWER:



```
# AES-like Block Cipher Modes
key = get_random_bytes(16)

# 16-byte Initialization Vector
iv = get_random_bytes(16)

# Plaintext
plaintext = b"Performance Analysis of AES Block Cipher Modes"
print("Original Plaintext :", plaintext.decode())

# ----- ECB MODE -----
cipher = AES.new(key, AES.MODE_ECB)

start = time.time()
ciphertext_ecb = cipher.encrypt(pad(plaintext, AES.block_size))
end = time.time()

print("\n==== ECB MODE =====")
print("Ciphertext:", ciphertext_ecb.hex())
print("Encryption Time:", (end-start)*1000, "ms")

cipher = AES.new(key, AES.MODE_ECB)
decrypted = unpad(cipher.decrypt(ciphertext_ecb), AES.block_size)
print("Decrypted Text:", decrypted.decode())

# ----- CBC MODE -----
cipher = AES.new(key, AES.MODE_CBC, iv)

start = time.time()
ciphertext_cbc = cipher.encrypt(pad(plaintext, AES.block_size))
end = time.time()
```

Console Output:

```
Run started
Initializing environment
Installing packages
Running code
Original Plaintext : Performance Analysis of AES Block Cipher Modes

==== ECB MODE =====
Ciphertext:
9d2531c88879bed91882d737b09fc6507fd3e6e835b3009d06f11c99e3deb43db2fd4c210a750b303e8830b9530ca1d2
Encryption Time: 0.9999275207519531 ms
Decrypted Text: Performance Analysis of AES Block Cipher Modes

==== CBC MODE =====
Ciphertext:
1c615eb0e8c792c7c0f3443a4c1999d5be81800f6cf6d060fa39492873bacc2b3b234e2444a6af2b0a579e3945a07bf
Encryption Time: 0.0 ms
Decrypted Text: Performance Analysis of AES Block Cipher Modes

==== CFB MODE =====
Ciphertext:
a7942e1ab18cfe9af6bec78e3b7f2aa9294be81cb0bcd74834143860ff25759e4253c86c878eb17751e45b0e764
Encryption Time: 0.0 ms
```

3. Debugging RSA Cryptosystem Implementation

A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

ANSWER:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import time

# Generate RSA Key Pair
start = time.time()

key = RSA.generate(2048)

private_key = key
public_key = key.publickey()

end = time.time()

print("RSA Key Pair Generated Successfully")
print("Key Generation Time:", (end-start)*1000, "ms")

# Plaintext
plaintext = b"Secure RSA Encryption"

print("\nOriginal Plaintext:")
print(plaintext.decode())

# Encryption
cipher_encrypt = PKCS1_OAEP.new(public_key)

start = time.time()

ciphertext = cipher_encrypt.encrypt(plaintext)

end = time.time()
```

Console

RSA Key Pair Generated Successfully
Key Generation Time: 3059.000015258789 ms

Original Plaintext:
Secure RSA Encryption

Encrypted Ciphertext (Hex):
380e773f1c45b72f5f1f9f8d208b025451d691807308f48c5c3611c4cdcd8fc38550c0e52b993f634a563fee1e09ab4c655ba9fa0c05284312c001455f16b075f8b983718411cfa5a9f57349b23aac8f51a0a4cc3a99199e13d4ca32c7bd3624a60554b85366b54d63ad5d5e314f15660f173199c36f9fe55c9440d08e1c8ddd062c64b5320e1d31aa84bb5656531879416aca09c7ad191e8e8af1df63568e74426ddf01880f4a55597ddd1bebd469b691b159ba38377a9c536765889616fcb45fea36c67ab95dc567dbf3c506783bac736dcc5b0db0053d25c5b8e3e08eeac0e31ed84e53323d5e239a8bdd4f2ba93799344afc4c22cc541d1d82ddcae0

Encryption Time:
1.9998550415039062 ms

Decrypted Plaintext:
Secure RSA Encryption

Decryption Time:
12.000322341918945 ms

Run completed in 5193.399999976158ms

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the Diffie-Hellman key exchange is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.

B. Optimize exponentiation operations for large prime values.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

ANSWER:

```
import time

# Public Parameters
p = 23      # Prime Number
g = 5      # Generator

# Private Keys
private_key_A = 6
private_key_B = 15

print("Prime Number (p):", p)
print("Generator (g):", g)

# Generate Public Keys
start = time.time()

public_key_A = pow(g, private_key_A, p)
public_key_B = pow(g, private_key_B, p)

end = time.time()

print("\nPublic Key of User A:", public_key_A)
print("Public Key of User B:", public_key_B)

print("\nPublic Key Generation Time:", (end-start)*1000, "ms")

# Shared Secret Generation
start = time.time()

shared_key_A = pow(public_key_B, private_key_A, p)
shared_key_B = pow(public_key_A, private_key_B, p)

end = time.time()
```

Console

Prime Number (p): 23
Generator (g): 5

Public Key of User A:
8

Public Key of User B: 19

Public Key Generation Time:
0.0 ms

Shared Secret Key (User A):
2

Shared Secret Key (User B): 2

Shared Key Generation Time:
0.0 ms

Key Exchange Successful!
Secure Shared Secret = 2

Security Enhancement:
Use Digital Signatures or TLS Certificates

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing Elliptic Curve Cryptography for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.

B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

ANSWER:

```
from Crypto.PublicKey import ECC
from Crypto.Hash import SHA256
import time

# Generate ECC Key Pair for User A
start = time.time()

key_A = ECC.generate(curve='P-256')
public_A = key_A.public_key()

# Generate ECC Key Pair for User B
key_B = ECC.generate(curve='P-256')
public_B = key_B.public_key()

end = time.time()

print("ECC Key Generation Successful")
print("Key Generation Time:", (end-start)*1000, "ms")

# ECDH Shared Secret Generation
start = time.time()

# User A computes shared secret
shared_A = public_B.pointQ * key_A.d

# User B computes shared secret
shared_B = public_A.pointQ * key_B.d

end = time.time()

print("\nShared Secret Generated Successfully")
```

Console

ECC Key Generation Successful
Key Generation Time: 0.9999275207519531 ms

Shared Secret Generated Successfully

Shared Secret (User A):
a89e6dad5061b338d83baa2329bd02bfc9d4a59d5bdd69395642c50c9875978

Shared Secret (User B):
a89e6dad5061b338d83baa2329bd02bfc9d4a59d5bdd69395642c50c9875978

Shared Secret Generation Time:
3.999948501586914 ms

Key Exchange Successful!

Comparison with RSA

ECC Key Size : 256 bits
Equivalent RSA Key Size : 3072 bits

ECC provides:

- Faster Key Exchange
- Lower Memory Usage